

JobFinder Internals

A local-first job-application assistant: nineteen-plus scrapers, a deterministic fit model, an embedding-backed answer bank, and an auto-apply engine whose central rule is that it will stop rather than state something about you that you never said.

RUNTIME	STORAGE	AUTOMATION	MODELS
Node 20+ · Next.js 14 App Router	SQLite (WAL) or TimescaleDB	rebrowser-playwright over CDP	Ollama · qwen2.5:7b-instruct · all-minilm
NETWORK BOUNDARY	SOURCE SIZE		
Job boards + cited sources only	≈ 20,800 lines across lib/, connectors/, app/		

CONTENTS

01 The constraint	09 Auto-apply
02 System map	10 Derived answers
03 Storage	11 Anti-detection
04 Acquisition	12 Interview prep
05 Fit scoring	13 Scheduler
06 Retrieval	14 Pipeline & mail
07 LLM layer	15 Packaging
08 Autofill	16 Where the ML is

SECTION 01

The constraint that shapes everything

Every architectural decision in this codebase descends from one commitment: the machine holds the CV, the answers, the cookies and the database, and nothing else does. There is no server, no account, and no telemetry.

That is not a privacy footnote bolted onto a normal web app — it changes what is buildable. A cloud job-application tool would score postings with a frontier model, keep a shared embedding index, and log every field it filled. None of those are available here. Instead:

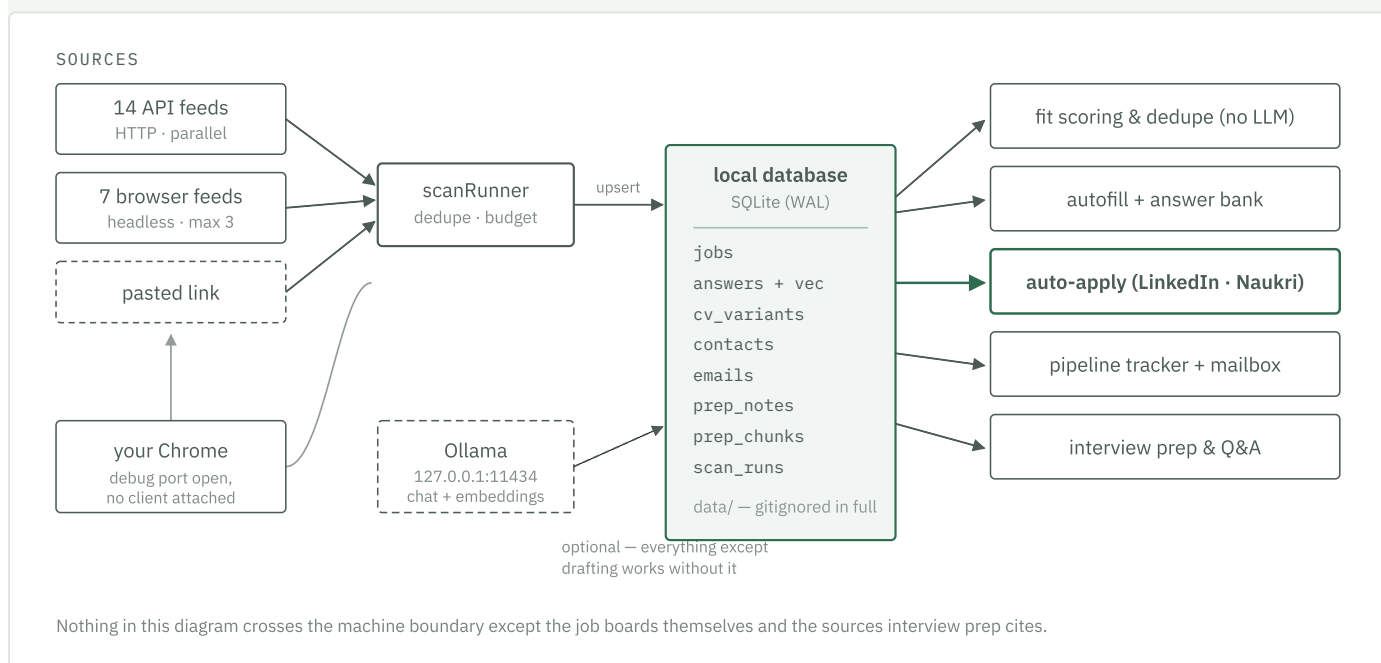
- **Scoring must be deterministic and CPU-bound.** Running an 8B local model over 347 scraped postings would take hours and give different answers each pass, so ranking is a weighted rule model that finishes in well under a second (§5).
- **Retrieval must work on a 46 MB embedding model.** The answer bank is matched by cosine similarity over `all-minilm` vectors, and the choice of model was settled by measurement rather than reputation (§6).
- **The LLM is the last resort, not the first.** It drafts prose. It is never allowed to supply a fact about the candidate (§9).
- **The browser is the user's real Chrome.** A fresh bundled Chromium with no history is a worse bot-detection outcome than an installed browser with a lived-in profile, so the code hunts for a real one first (§11).

The second shaping fact is that this software presses buttons an employer sees. Auto-apply is the only subsystem that produces an irreversible external effect, and it is written defensively to a degree that the rest of the codebase is not: dry run by default, arming never remembered between runs, a daily cap derived from the database rather than an in-memory counter, and a hard rule against inventing an answer.

SECTION 02

System map

Five stages, one database. Everything crosses the same `jobs` table, which is why a posting pasted from a link is indistinguishable downstream from one that came out of a scraper.



The pipeline. The database is the only integration point; each subsystem reads and writes it and knows nothing about the others. That is why a pasted link becomes a first-class job with no special-casing downstream, and why the scheduler can drive a scan and an apply run through exactly the same code the UI buttons use.

Storage: one facade, two engines

`lib/db.js` exposes three functions — `all()`, `get()`, `run()` — and hides whether the rows come from a local SQLite file or a Postgres/TimescaleDB instance named by `TIMESCALE_URL`. Route handlers never branch on backend.

The facade is thin, but three details in it are load-bearing:

Placeholder translation

SQL is written with SQLite's `?` and rewritten to `$1`, `$2...` for Postgres by a positional regex replace. One dialect in the source, two at runtime.

Type-parser pinning

`node-postgres` returns `BIGINT` and `NUMERIC` as strings to avoid precision loss. Every bigint here is an epoch-millisecond timestamp or a count, all comfortably inside `Number.MAX_SAFE_INTEGER`, so both OIDs are re-registered to parse as numbers. Without that, timestamps arrive as `"1717000000000"` and every date comparison silently misbehaves on one backend only — the classic "works on SQLite, breaks on Timescale" bug.

`run()` returns `{ changes }`

Callers need to distinguish an insert from a row skipped by `ON CONFLICT DO NOTHING`. Without it, a source returning fifty already-known jobs and a source returning nothing reported identically.

Migration strategy: additive ALTERs in a try block

There is no migration table and no version number. Schema evolution is a list of `ALTER TABLE ... ADD COLUMN` statements each wrapped in `try { } catch { }`, run on every boot. If the column exists the statement throws and is swallowed; if it does not, it is added. Postgres gets the same list written as `ADD COLUMN IF NOT EXISTS`.

For a single-user local app this is the right trade: it is idempotent, it cannot half-apply, and a user upgrading from any past version converges to the current schema in one boot. It does not support renames or type changes — but no shipped version has needed one.

Two repairs also run at boot and are worth noting because they show the class of bug that survives in a local-first app: a loop that peels double- and triple- `JSON.stringify`-ed `profiles.filters` values left by an earlier PUT handler, and a one-line `UPDATE` that restores answers stranded at `status = 'needs_input'` while holding a real value. In a hosted app you would fix the data with a one-off script. Here the fix has to ship inside the product, because you cannot reach the database.

Schema shape

TABLE	CARRIES	NOTABLE COLUMNS
jobs	Every posting ever seen, plus its pipeline state	canonical_key (cross-board dedupe), fit_score/fit_json, apply_kind, auto_apply_state, auto_apply_attempts, auto_apply_blocked_on
answers	The answer bank — one row per canonical question	embedding (JSON float array), status ∈ {pending, approved, rejected, needs_input}, options, asked_by
cv_variants	Several CVs plus their extracted text	cv_text — scored against each posting at apply time
prep_sources / prep_notes / prep_chunks	Raw research material → synthesised notes → embedded slices	evidence (verbatim JD sentence), demand, sources_json
prep_terms	Cached "is this a real technology?" verdicts	Global, not per-profile — the answer does not vary by user
contacts	Hiring addresses the ads printed themselves	context — the sentence it appeared in
emails	Job-related mail, matched and classified	asks (JSON array of what HR wants), suggested_status

The tables that carry state. Everything else is derived at read time.

The separation of `answers.embedding` from `prep_chunks.embedding` is deliberate and repeated in the code comments: one holds facts about the candidate used to fill forms, the other holds study prose. Merging them would let essay text leak into a form field and let "notice period: 30 days" surface as an answer to a systems-design question.

SECTION 04

Acquisition: the connector layer

A connector is a module exporting `{ id, label, loginUrl, requiresAuth, requiresBrowser, scan(ctx, profile) }`. Adding a source is one file plus one import. The registry is ordered by reliability — API sources first, login-gated scrapers last — because that ordering is also the order they appear in the UI and the order a user should trust them in.

Two pools, two concurrency policies

The scan runner splits targets by `requiresBrowser` :

- **API connectors** are pure HTTP against unrelated hosts, so all of them run in parallel with `Promise.all` . Budget: 45 s each.
- **Browser connectors** share one off-screen Chrome per profile. Tabs are cheap but too many simultaneously starves the renderer, so a hand-rolled worker pool caps in-flight scans at three. Budget: 120 s each.

Both budgets are enforced with `Promise.race` against a timer, not with per-request timeouts. The reason is specific: one wedged source used to stall the whole scan, because LinkedIn and Naukri could

each sit in a 90-second CAPTCHA wait and simply never report.

MEASURED

The single worst latency bug in this layer was not in the scan loop at all — it was `safeText()` calling Playwright's `innerText()` with no timeout. A selector that no longer matched fell through to Playwright's 30-second actionability default. At exactly **30,002 ms per missing field** and four to five fields read per card, the very first card burned the connector's whole 120-second budget, and the scan reported "timed out" when the truth was "the site changed its markup". The fix is a 2.5-second field timeout: far longer than a present element ever needs, short enough that a whole page of dead selectors costs seconds.

Reporting three numbers, not one

Connectors opt into `withFetched(jobs, n)`, which attaches a non-enumerable `fetched` count to the returned array. The scan then logs *fetched*, *matched* (survived keyword filtering) and *found* (actually new rows). A bare zero is ambiguous in three different ways — the source is broken, your keywords excluded everything, or you already have every job it offers — and only one of those is actionable.

Walls: sign-in versus block

`detectWall(page)` classifies an obstruction as `login`, `captcha` or `blocked`, and connectors throw a tagged `wallError`. The scan runner raises a visible sign-in window for the first two and deliberately does *not* for the third — you cannot click your way out of an IP ban, so a window would be pure interruption. It also caps attention at **one window per scan**: an expired LinkedIn cookie previously opened three windows (`linkedin`, `linkedinposts`, `naukri`) when one sign-in fixes all of them.

Cross-board de-duplication

The same role scraped from four boards collapses to one entry via a canonical key computed at insert time:

```
normalizeCompany("Acme, Inc.") → "acme"           // legal-form noise
stripped
normalizeTitle("Senior Backend Engineer (m/w/d) – Berlin")
                                → "senior backend engineer"
canonicalJobKey(job)            → "acme::senior backend engineer"
```

The function returns an empty string when there is not enough signal — a missing company, or a title under four characters after normalisation. Callers then treat the row as unique. Merging two genuinely different jobs is a worse error than failing to merge two identical ones, so the ambiguous case falls to the safe side.

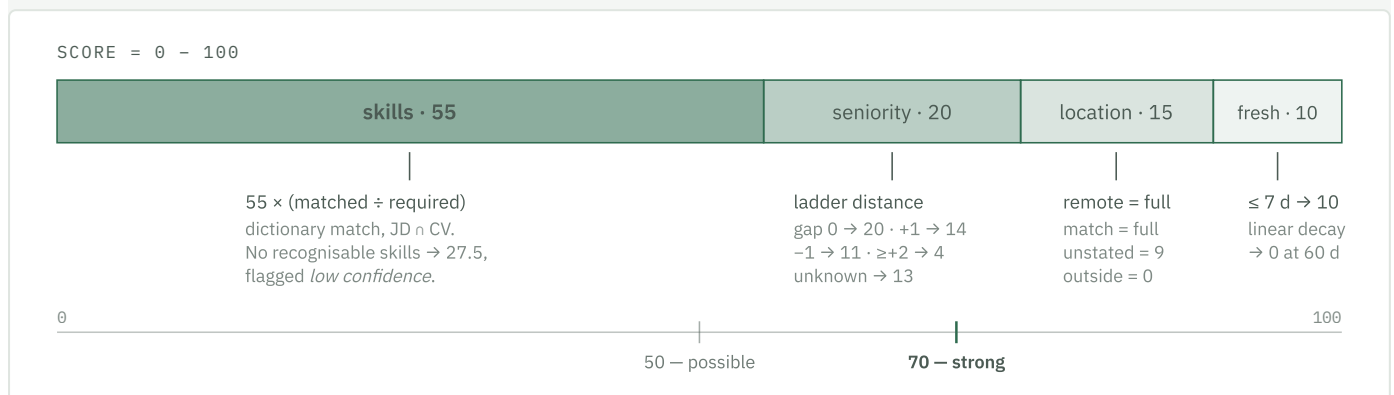
Learning the apply type during the scan

Most Naukri results are "Apply on company site", which auto-apply can do nothing with. Discovering that by opening each posting costs about five seconds a job. Instead the Naukri connector attaches a `page.on('response')` listener to the search page's own `/jobapi/v*/search` call — a request the page makes anyway — and reads `companyApplyJob` straight out of the JSON. No extra request, no header guessing, and nothing to keep in sync when the card markup changes. That flag lands in `jobs.apply_kind`, and external postings never enter a batch at all.

SECTION 05

Fit scoring, deterministically

Fit turns an undifferentiated pile of postings into a ranked shortlist. It is a weighted linear rule model over four components — no learning, no model file, no network.



The weights are the argument. Skills carry more than half the score because a mismatch there is the one thing that reliably ends an application; freshness carries only ten because a stale posting is a lower-probability reply, not a wrong role. Bands are advisory labels over the same number, not a separate classifier.

Confidence is reported separately from score

This is the design detail that makes the number usable. `scoreJob` returns `{ score, band, matched, missing, reasons, confidence }`, and `confidence` answers a different question: *how much did we actually know?*

- No CV text over 200 characters $\rightarrow$ `low`, whatever the score says.
- No recognisable skills in the posting $\rightarrow$ `low`, and the skills component is awarded a neutral 27.5 rather than zero. Punishing a job for a thin description would rank a well-written ad for a bad role above a terse ad for a good one.
- Fewer than three detected requirements $\rightarrow$ `medium`.

Every score also ships its own explanation: the list of requirements you meet, the list you miss, and four plain-language reason strings. The stated principle is that a number you cannot interrogate is worse than no number.

The seniority ladder scans highest-first

Six ranked levels — intern, junior, mid, senior, staff, executive — each with a regex covering English and German phrasing. Detection iterates from *executive downwards* so that "Senior Staff Engineer" resolves to staff, not senior. "Head of X" is classified as executive rather than staff for a consistency reason: otherwise "Head of Engineering" and "Head of Machine Learning" would land on different rungs, which is indefensible to a user reading the explanation.

The candidate side

`candidateSkillProfile()` unions three evidence sources — extracted CV text, the profile bio and keywords, and every saved answer rendered as `"label: value"` — then runs the same dictionary matcher used on the job description. Using one matcher on both sides is what makes the comparison meaningful: the "Kubernetes" that puts a topic on your study list is the same token matched against your CV.

Freshness has a basis, not just an age

`lib/freshness.js` parses five distinct `posted_at` shapes observed live in the database — ISO with offset, ISO with Z, bare ISO, RFC-2822, and relative text like "3 days ago" or LinkedIn's compact "2h" — and rejects implausible values (before 2000, more than a day in the future). About **24%** of rows carry no published date at all, because Naukri, LinkedIn, Y Combinator and Wellfound do not print one on their listing pages.

So `freshnessOf()` returns `{ at, basis, ageDays }` where `basis` is either `'posted'` or `'seen'`. Claiming "posted today" when the source never said so is a claim the system cannot support, and acting on a wrong posted date costs exactly the first-mover advantage the feature exists to provide. The `"mo"` unit is tested before `"m"` in the relative-time regex, or months would be read as minutes.

SECTION 06

Retrieval: the embedding decision

This is the one place in JobFinder where a model choice was settled by running an experiment, and where the obvious default turned out to be actively harmful.

What retrieval has to do

Application forms ask the same twenty facts in a hundred phrasings. "Total years of experience", "Experience (years)", "How many years have you worked in this field?" are one question. Exact-key matching handles the identical case; a canonicaliser (`lib/answerBank.js :: canonicalKey`) collapses seventeen common families by regex; everything else needs semantic similarity.

So each answer row stores a vector of `label + "\n" + value`, embedded at write time. At read time the incoming field label is embedded and cosine-ranked against every approved row.

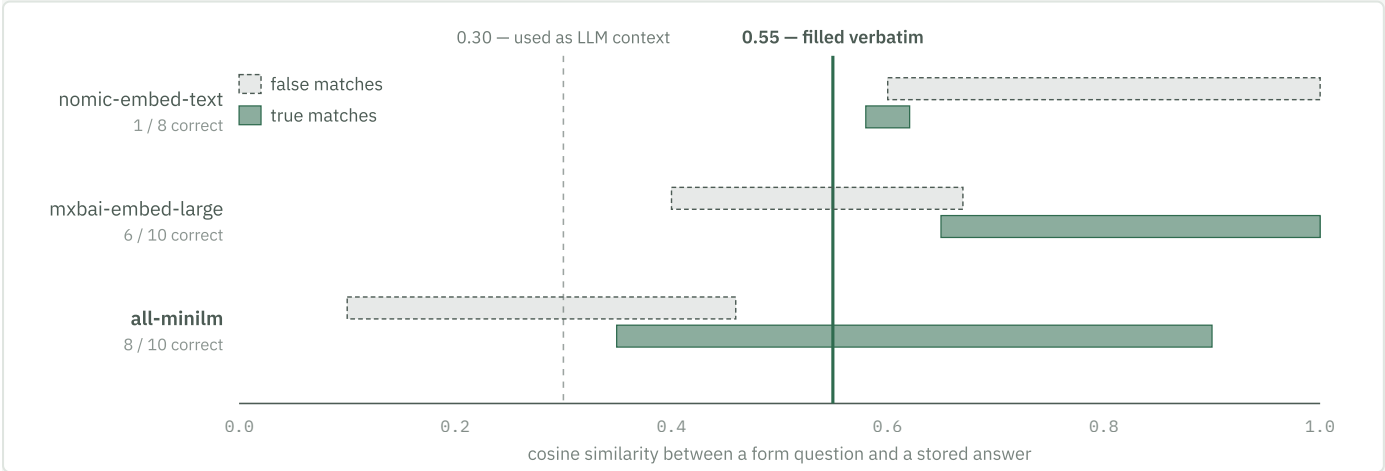
Why `nomic-embed-text` failed

The project shipped with `nomic-embed-text` , the obvious default. Its vectors turn out to be dominated by a single component — roughly -3.7 where every other dimension is around 0.1 — and that component barely moves with the text. Cosine similarity is then measuring almost nothing but that constant. On the real answer bank it scored the unrelated pair *"Expected CTC"* against *"City: Berlin"* at **1.000**, and got 1 of 8 answerable questions right.

That is not a quality gap; it is a failure mode with a direction. High-confidence false matches get written into forms, and because the observer records what lands in a field, a wrong guess was then learned back into the bank as an exact-key answer — which the next run filled with total confidence, never reaching the vector stage at all. One bad guess became twelve bad rows. (The fix for that second half is in §8.)

EMBEDDING MODEL	CORRECT	TRUE-MATCH COSINE	FALSE-MATCH COSINE	VERDICT	SIZE
<code>nomic-embed-text</code>	1 / 8	0.58 - 0.62	0.60 - 1.00	Overlapping	274 MB
<code>mxbai-embed-large</code>	6 / 10	0.65 +	up to 0.67	Overlapping	670 MB
<code>all-minilm</code>	8 / 10	0.35 - 0.90	up to 0.46	Separated	46 MB

Seventeen real form questions against a 35-row answer bank. "Separated" means every false match scored below every threshold at which the system fills verbatim.



Separation, not absolute score, is what matters. `all-minilm` 's cosines run *lower* in absolute terms than the alternatives — but every false match lands below 0.46, so the 0.55 verbatim-fill line has nothing wrong above it. With the other two, false matches sit on the wrong side of that line, which is how *"India"* ended up typed into an *Expected CTC* field.

COUPLED CONSTANTS

Because `all-minilm` 's scores run lower, `SIM_THRESHOLD` in `lib/knowledge.js` was retuned from 0.45 to **0.30**. Genuine matches in this bank go as low as 0.35 (*"Which university did you attend?"* → school). The two constants are a pair: changing the embedding model without re-measuring the threshold silently breaks retrieval, so the source says so at both sites.

Two thresholds, two jobs

0.30 — `SIM_THRESHOLD` , `knowledge.js`

The bar for *being shown to the model as grounding context*. Low, because a loosely related answer still helps the LLM avoid inventing, and the LLM can decline.

0.55 — `SEMANTIC_CONFIDENT` , `autofill.js` and `autoApply.js`

The bar for *typing a stored value into a form verbatim, unsupervised*. Deliberately not lowered for auto-apply, which has *less* human oversight, not more.

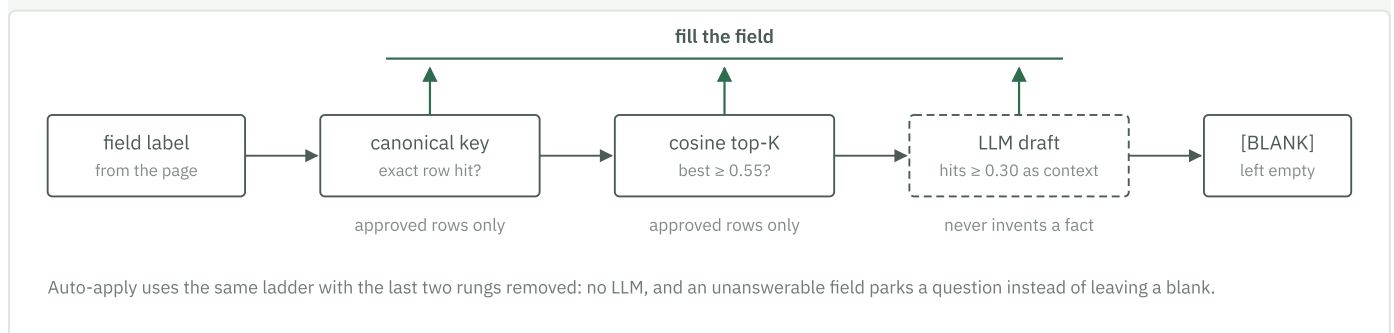
0.52 — `prep KB` , `kb.js`

A third, independently tuned bar for the study-notes Q&A. Measured against its own corpus: off-topic questions ("how do I cook risotto?") still reach 0.35–0.45 against study prose, so 0.35 would have let them claim to be grounded.

Self-healing on a model change

`cosine()` returns 0 on a length mismatch, so after an embedding-model swap every stored vector silently scores zero and the bank goes quiet rather than wrong. That is the safe failure, but it is still a failure. `backfillEmbeddings()` therefore probes the live model for its dimension, compares it against every stored vector's length, and re-embeds anything that does not match. The answer bank is the one piece of user data an upgrade must never invalidate.

The retrieval ladder



Cheapest, most certain first. Each rung is strictly more expensive and strictly less certain than the one before it. The gate on rungs one and two is `status = 'approved'` , applied inside `retrieve()` so that it covers every consumer at once — an unreviewed answer can neither be typed into a form nor quoted back by the model.

SECTION 07

The language-model layer

`lib/llm.js` is a thin Ollama client with four opinions baked into it, each traceable to a failure.

Instruct models, not reasoning models

The project originally used `deepseek-r1:8b`. Reasoning models emit a `<think>` block before any answer. Autofill asks for short answers at `num_predict: 80` — so the entire budget went into thinking, `stripThinking()` removed all of it, and the function returned an empty string. Not slow: *silently failing*.

MODEL	LATENCY	RESULT
deepseek-r1:8b	19.3 s	All four empty
qwen2.5:7b-instruct	2.9 s	All four correct
qwen2.5:3b-instruct	3.1 s	Two wrong — invented a salary figure

Four ordinary short fields — notice period, years of experience, work authorisation, expected CTC — at `num_predict: 80`.

The 3B was no faster once warm and materially less accurate, which is the wrong trade for text that goes into a real application. Using the same model for chat and for bulk synthesis also means *one* model resident instead of two — on a 16 GB machine, deepseek (5.2 GB) plus qwen (4.7 GB) pinned for thirty minutes alongside Chrome is what produced an Ollama stall during development.

When a response strips down to nothing but was non-empty before stripping, the client logs a loud, specific warning naming `num_predict` and `done_reason` — because that failure is invisible at the call site and previously surfaced as "skill extraction returns zero results".

Every call has a deadline

A request Ollama accepts but never generates for hangs forever, and anything awaiting it hangs with it. Observed during a prep build: two connections open to `:11434`, the model loaded, the runner at 3.7% CPU — not generating, not erroring. `DEFAULT_TIMEOUT_MS` (240 s, overridable) turns that into an error. Critically, **a timeout is not retried**: transient network errors get exponential backoff at 1 s / 2 s / 4 s, but a wedged model is surfaced with a message naming the fix, because hammering it again just burns another four minutes.

Model resolution is capability-based

`resolveChatModel()` and `resolveSynthModel()` follow the same shape: an explicit per-profile setting wins; otherwise the best installed instruct model from a preference list; otherwise the default tag. The installed-model set is cached for 60 seconds off `/api/tags`. This means installing a larger instruct model is all it takes to speed up prep, and a fresh machine with some other model does not fail on the first autofill by requesting a tag it does not have.

Pinned to IPv4, and kept warm

The default URL is `http://127.0.0.1:11434`, not `localhost`. Node 18+ resolves `localhost` via DNS and on some macOS configurations reaches `::1` first, where Ollama — which binds 127.0.0.1 by default — is not listening, producing a cryptic `fetch failed`. `keep_alive: '30m'` keeps the model resident between calls so the first real autofill after a pause is not a cold load.

Output hygiene

Asked to write a form answer, an instruct model opens with *"Certainly! Here's a detailed response to describe a project..."* — and that sentence would be typed straight into an employer's textarea.

`stripPreamble()` removes a leading conversational wrapper, but only under guards worth stating because the naive version destroyed data:

- The preamble regex terminates on `:` or `!` and **never on** `.`. An earlier version allowed `.` with a greedy run before it, so the match ran to the *last* full stop and swallowed the whole answer.
- If stripping leaves fewer than 20 characters, the original is kept. Trimming an answer to nothing is worse than leaving the preamble in.
- At most two passes, then a leading markdown heading is removed — the same problem in a different costume.

Structured output uses Ollama's `format: "json"` to constrain decoding, plus a hand-rolled `extractJson()` that peels code fences and then brace-matches from the first `[` or `{`. That function appears in five modules; it is duplicated rather than shared because each copy is small and the modules are otherwise independent.

SECTION 08

Autofill and the observer

The observer (`lib/observer.js`) is a self-contained IIFE injected into every page and frame. It cannot import modules — it runs inside a hostile page — so it carries its own copies of the canonicaliser and the label walker. `scripts/check-canon-parity.mjs` asserts that its `CANON` table matches the server-side one, so drift fails a check instead of silently splitting the bank in two.

What it refuses to touch

Three categories are excluded from both filling and learning:

- **Honeypots** — measured, not guessed. A field is a trap if it is `type=hidden`, under 2 px in either dimension, positioned off-screen, `display:none` / `visibility:hidden` / opacity under 0.05, `aria-hidden`, or has no `offsetParent` while not fixed. Completing an invisible field is a self-inflicted bot verdict — it is the one field on a page a real user cannot fill.
- **CAPTCHAs** — unanswerable, and an answer poisons the bank.
- **The board's own search box** — it sits on the same page as the application but describes what you were looking for, not who you are.

Sensitive types (`password`, `cvv`, `card`, `ssn`, `secret`, `otp`, `verif`) are excluded at the point of capture, before anything is stored.

The feedback loop, and closing it

THE BUG THIS PREVENTS

`setNativeValue` dispatches `input` and `change` so the host page's framework notices the write. *The observer listens to those same events.* So every value autofill guessed was immediately recorded as though the user had typed it. A weak semantic guess — "certifications" → "Kolkata" — became an exact-key answer, which the next run filled with total confidence, never reaching the vector stage at all. One wrong guess turned into twelve bad rows.

The fix: tag anything written by the machine (`el.__jfMachine = value`) and refuse to learn from it. Only a human keystroke creates knowledge. If the user later edits the field, the value no longer matches the tag and it is learned normally — the correction is exactly what should be captured.

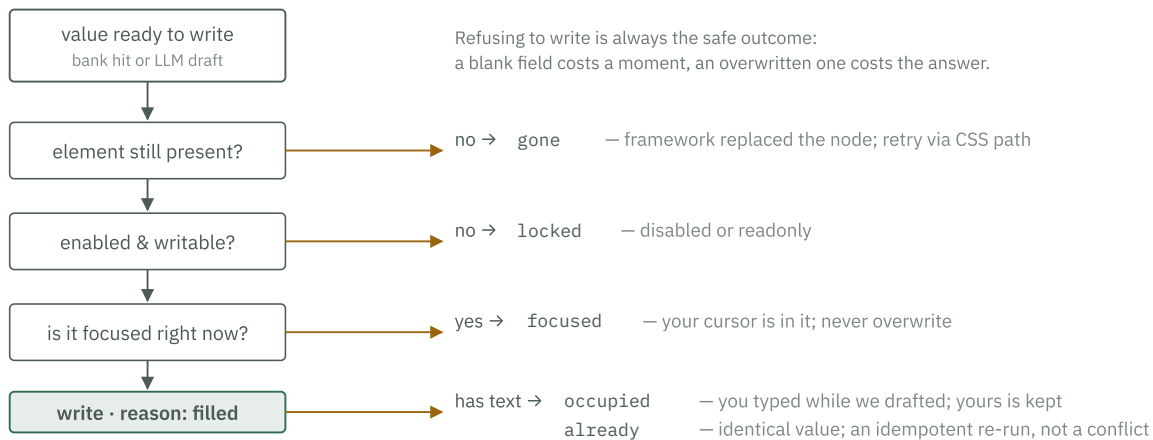
The offline buffer

`window.__jobfinderObserve` is a Playwright `exposeBinding` : it exists only while an automation client is attached. Under the lazy-attach model (§11) that is a roughly two-second window during Autofill — so everything typed *afterwards* (the corrections, the fields the LLM left blank, the answers written just before Submit) reached nothing and was lost.

But the observer is ordinary page JavaScript and keeps running with zero automation attached. It just needs somewhere to put what it sees. So every observation is also written to `localStorage` under `__jobfinder_buffer_v1` , last-write-per-key, capped at 200 entries; the next attach calls `__jobfinderDrainBuffer()` and clears it. Nothing about this weakens the detached-for-verification property — no CDP client, no network, purely a same-origin key/value write.

Write-time re-validation

There can be *minutes* between the scan that identified a field and the write that fills it: the LLM pass drafts up to twelve fields two at a time, and each draft is a full local generation. You keep typing during that wait. So `__jobfinderSetByPath` re-checks the live DOM rather than trusting the snapshot.



Every refusal has a distinct reason code, and `occupied` / `focused` surface in the log as "you typed *x* while I was drafting — kept yours, discarded mine". Distinguishing them from `gone` and `locked` is what lets the caller retry sensibly instead of guessing.

Frames, selects and concurrency

Autofill walks *every frame* of the target page. Workday, Greenhouse Embedded, Lever, iCIMS and SmartRecruiters all render the form inside one or more nested iframes; the top frame alone shows zero fields. Target selection defaults to the currently visible tab (`document.visibilityState === 'visible'`), falling back to the most recently opened page, because pressing Autofill while looking at step 3 of a form should fill step 3.

Concurrency is bounded by a tiny `pMap` : six parallel embedding lookups, two parallel LLM drafts (Ollama handles about two concurrent generations comfortably), and a hard cap of twelve LLM-filled fields per page so a giant form does not take forever.

`<select>` elements need their own path. Calling the `HTMLInputElement` value setter on one throws "illegal invocation", which an earlier catch swallowed — so dropdowns silently never filled. The observer now matches an option by value *or* visible text and sets `selectedIndex` ; if nothing matches it leaves the element alone rather than forcing a bad value, because setting a `<select>` to a value it does not have silently sets nothing.

The review queue

New captures land at `status = 'pending'` and are invisible to autofill until approved. Editing an approved answer sends it back to pending — the point of the gate is that nothing reaches a form without the user's say-so, including a changed value. A rejection is durable: re-harvesting the same page will not re-queue a value the user already rejected, or you would be rejecting the same row forever.

Beneath the queue sits a second line of defence, `junkReason()` , which rejects rows before they reach the bank at all. It is worth reading as a catalogue of how form-capture goes wrong: type/value contradictions (an "email" that is not an email, a "linkedin url" that is "Kolkata"), a one-word answer to a 60-character essay prompt, a paragraph answering a yes/no question — but only when the question really is yes/no, since "Have you worked with APIs, and if so in what context?" opens yes/no and then asks for prose — a placeholder used as a question ("Type your answer here" tells you nothing about what was asked), and a country stored as a dialling code.

Auto-apply: the refusal engine

This is the only subsystem that produces an irreversible external effect. Its central mechanism is not filling — it is stopping.

The rule

If the form asks something the answer bank cannot answer, the run stops. The LLM is not consulted. A drafted paragraph about why you want the job is a writing task and the model is good at it; *"how many years of Kubernetes do you have"* is a fact about you, and a model that has never met you cannot supply it. Getting that wrong is not a cosmetic bug — it is a false statement on a real application, sent under your name, that you cannot retract.

So `resolveAnswer()` has exactly two branches, `{ value }` and `{ missing: true }`. There is deliberately no third branch that asks a model.

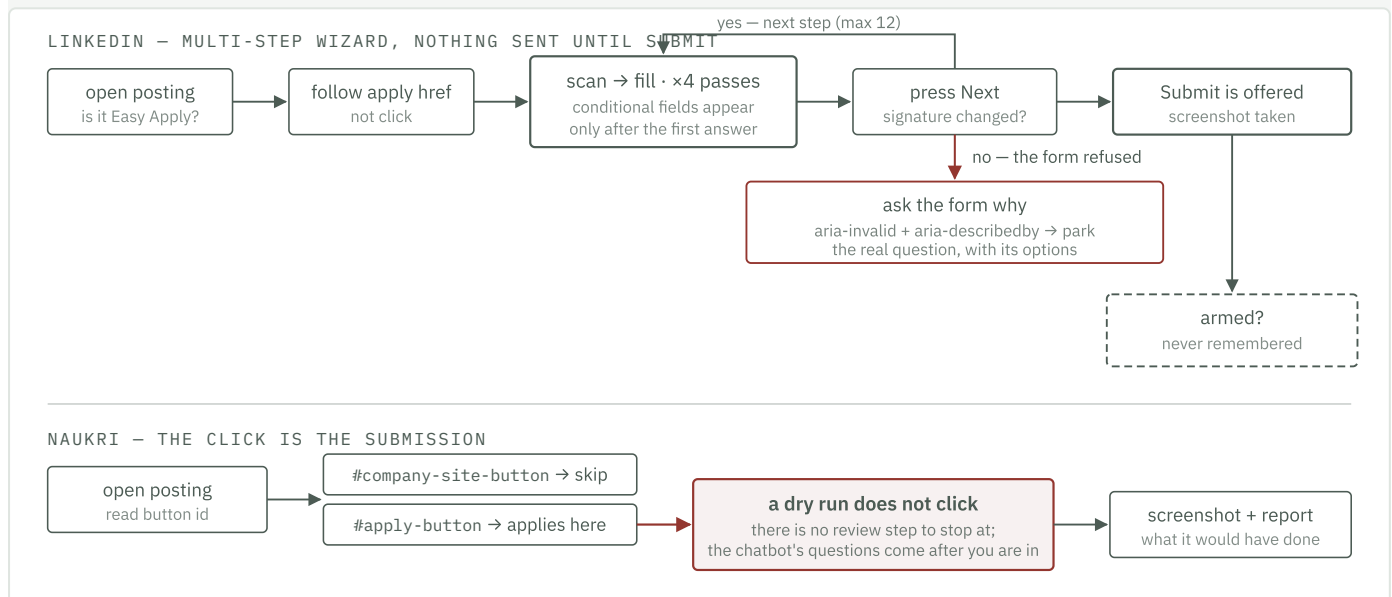
Parked questions, keyed like answers

An unanswerable field is written to the `answers` table with an **empty value** and `status = 'needs_input'`. Retrieval only ever reads `approved` rows, so a parked question can never be used as an answer — by autofill or by the model — until the user fills it in. Because it is keyed by `field_key` like every other answer, the same question asked by a second employer resolves to the same row: *answer it once, and every job waiting on it unblocks together*.

Three subtleties keep that loop from breaking:

- **needs_input means empty, always.** Re-parking clears the value. A row that keeps a stale value looks answered in the bank while retrieval ignores it — which is exactly how a question ends up permanently stuck.
- **The replace escape hatch.** Sometimes the bank *has* an answer that the form will not accept (a city for a dropdown listing only other cities). Without a flag, `parkQuestion` refuses it as already-known, the question never reaches the user, and every run keeps reporting it as the blocker.
- **Only invalidate what was actually attempted.** A field the form flags as invalid that we never wrote to is simply an empty required field — not evidence that the stored answer is wrong. Re-parking it unconditionally flips an answered row back to `needs_input`, where retrieval cannot see it, so the next run parks it again. The user fills it in, and the run quietly undoes that every time. The engine therefore tracks an `attempted` set per run and only invalidates keys in it.

Two sites, two different dangers



The asymmetry is the whole design. LinkedIn's wizard can be walked, inspected and backed out of, so a dry run fills everything and stops at Submit. Naukri's apply button *is* the submission, so a dry run must not click at all. Early-access roles ("Share Interest") are treated with the same care as a submission, because they are equally irreversible.

Reading refusal, because the site will not tell you

LinkedIn does not report that it refused to advance. If a required field is empty it simply does nothing when Next is pressed — same dialog, same percentage, no visible error. So the engine fingerprints the step (`stepSignature` : the progress percentage plus the ordered list of field ids/names/types) before and after the click. An unchanged signature *is* the refusal signal.

Two things follow from that. First, LinkedIn frequently omits `required` and `aria-required` on its custom widgets, so a field looked optional and got skipped — the refusal is the requirement signal, and anything still unanswered on that step is parked rather than skipped. Second, if *everything* is answered, the engine asks the form what it objects to by walking `aria-invalid` and `aria-describedby`. That distinguishes the two cases which look identical from outside: a field left empty, and a field filled with something the site rejects.

Filling that confirms it took

Writing `.value` is enough for an ordinary input and wrong for three widget classes:

- **Typeaheads.** LinkedIn's "Location (city)" keeps its real answer in React state, which only updates from genuine keystrokes plus a chosen suggestion. Write to it directly and the box shows your text while the form still considers it empty — so Next does nothing, forever, with no error. The engine types with `pressSequentially` at 90 ms per character and then clicks a `[role="option"]`.
- **Selects.** React-controlled dropdowns ignore a synthetic `change` : the box shows the new label while React's state stays on the placeholder. Playwright's `selectOption` drives it the way a person does.
- **Verification.** A `<select>` is confirmed by its *selected option's text*, not by `.value` — LinkedIn's option values are URNs while the visible text is "1 year", so comparing `.value` to what was asked reports failure on a dropdown that is set perfectly well, and the run then blames a field that is fine.

Every DOM read goes through `safeEval`, which swallows "Execution context was destroyed". Clicking Next or Submit tears the context down under any evaluate in flight; these reads are all best-effort by nature, so a destroyed context means "could not do it", not "abort the application".

Batch composition

A batch drawn purely by freshness ends up wherever the boards happened to be busy that morning, which is usually all one place. The mix is set deliberately instead: **50% home, 30% worldwide, 20% remote**, with quotas allocated by largest-remainder so the total always sums to the limit exactly rather than drifting a job short on odd numbers. Places a thin bucket cannot use are backfilled in priority order — *remote first, then home, then worldwide* — because Naukri's worldwide bucket is almost always short and those places are better spent on remote roles than on more local ones.

Bucketing puts remote ahead of geography on purpose: "Bengaluru (Remote)" is a remote job that happens to name a city, and treating it as domestic would let the home bucket swallow the remote quota.

Ordering: day buckets, then fit

MEASURED

Ordering by `fit_score` alone put a high-scoring 87-day-old Naukri listing ahead of a job scanned that morning. On a real ten-job batch ordered that way: **8 expired, 1 already applied, 1 external, 0 actually applied**. The whole budget went on dead postings.

The fix is to sort by *age in whole days first*, then fit within the same day. Sorting by fit across the whole window has the same failure in miniature, because fresh postings usually have no fit score yet — scoring runs later — so they sort as 0 and land behind three-week-old scored ones.

Slot accounting: a batch of ten means ten real attempts

The naive loop consumed one candidate per slot, so a batch of ten could finish having applied to nothing — every place taken by a posting that turned out to redirect to the employer, or to have expired. Those cost a few seconds and teach something permanent (`apply_kind` is recorded), so they must not count.

Each worker therefore *claims* a place before opening a posting and *hands it back* if the outcome was not a real attempt. Reserving up front matters too: without it, every worker sees `usable < limit` at the same moment and they all start one more, so a limit of 10 quietly becomes 12.

OUTCOME	CONSUMES A SLOT?	COUNTS AGAINST RETRIES?	RATIONALE
applied, dry_run, error, applied_incomplete	Yes	Only error	We opened a form and worked on it
needs_input	Yes	No	Waiting on the user is not a failure. Counting it threw away two good applications after three passes purely because nobody had typed an answer.
external, expired, already_applied	No	No	Decided in seconds without opening a form; apply_kind learned permanently
Transient (ERR_HTTP_RESPONSE_CODE_FAILURE, 429, timeouts)	No	No	The site refused <i>us</i> . The job is left completely unrecorded and still eligible, so a rate-limited afternoon cannot permanently discard good postings.
Outcome classes and what each one costs.			

After three usable attempts still ending in `error`, a posting is `abandoned`. That is per *posting*, never per role: a reposted opening is a different row with its own `external_id` and its own counter, so giving up on today's broken form does not stop an application when the same employer posts it again next month.

Parallel across boards, serial within one

LinkedIn and Naukri are unrelated servers with unrelated rate limits, so running both at once is free speed — the run takes as long as the slower board rather than the sum. Firing ten applications at the *same* board simultaneously is not free: it is the clearest automation signal either site gets, and the penalty is a restricted account, which costs the logged-in session everything else here depends on.

So an armed run is one-at-a-time per board with a randomised 12–30 second gap between submissions. A dry run submits nothing, so it opens a few tabs at once — but not the same number for both boards: three concurrent LinkedIn tabs produced `ERR_HTTP_RESPONSE_CODE_FAILURE` on half a batch, so `DRY_WIDTH` is `{ linkedin: 2, naukri: 3 }`.

Not reopening jobs that cannot progress

A job parked on an unanswered question is not worth reopening: the run would walk the same wizard, hit the same blank field, park it again, and spend a slot to learn nothing. So eligibility joins `jobs.auto_apply_blocked_on` against the set of keys currently at `needs_input` and excludes anything still waiting. The job returns automatically the moment the question is answered — which is the entire point of keying questions by `field_key`.

SECTION 10

Answers that are computed, not stored

Two classes of question have a correct answer that a stored string gets wrong. These are the only cases where auto-apply fills a field without a human having typed that exact value — and in both, the user

supplied the underlying fact and the system is only doing arithmetic.

Notice period

If your last working day is fixed, the honest answer shrinks by one every morning. Storing "45 days" makes it wrong tomorrow and badly wrong in six weeks — and wrong in the direction that matters, because a recruiter plans around the date you gave them. So the source of truth is the *date*, entered once, and the number is derived at the moment a form asks.

The interesting part is dropdowns. Forms express notice as buckets — "Immediate", "15 days", "1 month", "60-90 days", "More than 90 days" — and `matchNoticeOption()` parses each label into a `[lo, hi]` day range, then:

1. An exact containing bucket wins.
2. Otherwise, the **smallest option still greater than or equal to** the real figure. Rounding *up* is deliberate: telling an employer you can start sooner than you can is the error that costs you the offer.
3. If every option on offer is shorter than the truth, it returns `null` and falls through to the ordinary bank — say nothing rather than something false.

ONE REGEX, SIX WEEKS OF WRONG ANSWERS

The "Immediate" pattern needs `\b0\s*days?\b`, not `0\s*days?`. Without the word boundary, "0 day" matches the tail of "3-0- days" — so "30 days", "60 days" and "31-60 days" all parsed as Immediate, and a 45-day notice period would have been reported to employers as "available now". `scripts/test-notice-period.mjs` exists specifically to guard this arithmetic.

Salary expectation, converted by purchasing power

Your figure is one number in one currency. A form in Berlin wants euros, one in Dubai wants dirhams, and typing `3000000` into either is a number so wrong it reads as a mistake rather than an ask. But *which* conversion you use changes the answer enormously, and both are defensible:

BASIS	WHAT IT ANSWERS	RESULT	CONSEQUENCE
Market exchange rate	What your money is worth if you exchange it	≈ \$36,000	Nobody in the US is hired at that. Underprices you threefold, and it is very hard to walk back.
Purchasing power (PPP)	What it takes to live the same way there as here	≈ \$130,000	The honest basis for "what would I need to move" — which is what the question is really asking.

₹30,00,000 expressed for a US employer, two ways. The difference is a factor of 3.6.

The conversion is `base ÷ PPP[home] × PPP[target]`, with factors from World Bank ICP (LCU per international dollar, 2024), rounded to something a person would actually type — roughly three significant figures on a magnitude-appropriate step, with a coarser step for JPY, KRW, IDR and VND. Both figures are logged every time, so the number is never a black box, and a *currency-specific answer you set yourself always wins* — an `expected salary usd` row in the bank beats anything computed.

Currency detection is ordered by strength of evidence: a currency code in the field label, then a currency symbol, then "lakh"/"LPA", then the job's country. A bare \$ resolves to *nothing* — it could be four different currencies, and the difference between USD and SGD is a third of the number. When no currency can be identified the field is not guessed at; it is parked as a question that names the problem: *"Expected CTC — which currency? the form does not say which currency it wants, and the job does not name a country."*

DESIGN NOTE

The PPP and FX tables are hard-coded, not fetched. A local-first app should not depend on a currency API to fill a form, and PPP moves slowly enough that a stale table is a rounding error rather than a wrong answer. FX is carried purely so the log can show the contrast; it is never the basis for what gets typed.

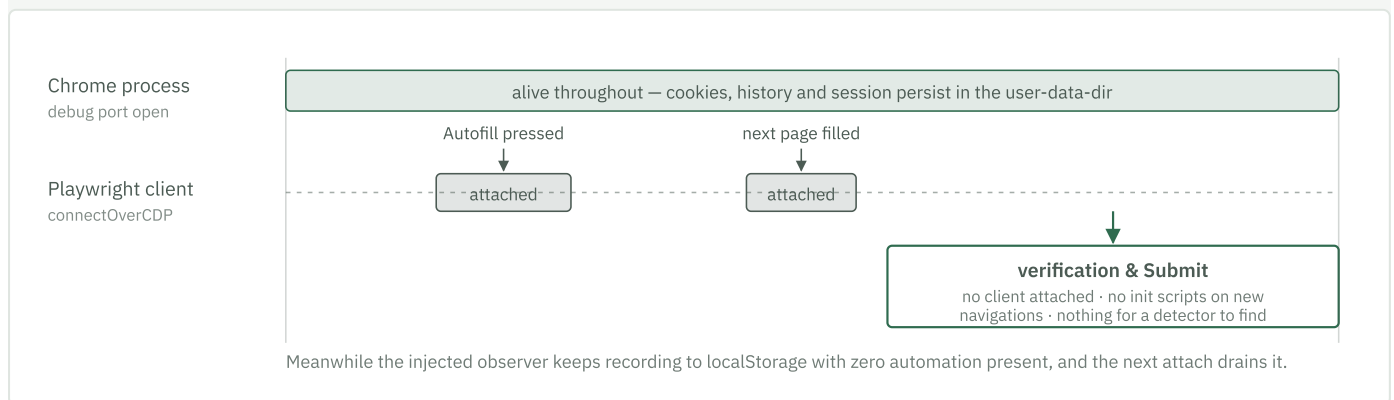
SECTION 11

Anti-detection and the lazy-attach model

Commercial bot detection scores two things: what the JavaScript environment looks like, and whether an automation client is currently driving the page. The stealth script addresses the first. Lazy attach addresses the second, and is the more interesting idea.

Lazy attach

Chrome does not set `navigator.webdriver = true` because a debugging port is open. It sets it when a CDP client actively issues `Runtime.enable`. And page JavaScript cannot probe `localhost:<port>` across origins. So a Chrome running with `--remote-debugging-port` and **no client connected** is fingerprint-identical to plain Chrome.



The window you get verified in has nothing attached to it. `withAttachedContext()` connects over CDP, re-injects the stealth and observer scripts into already-open frames (both are idempotent IIFEs), runs the callback, and disconnects — leaving Chrome alive. Pages loaded between attaches receive no init scripts at all, which is precisely what you want for the human-verification step that follows an autofill.

Because the in-memory port map is lost whenever Next.js restarts the server process — which it does on almost any edit — while *your Chrome keeps running*, the debug port is also written to disk (`.jobfinder-cdp.json`) and rediscovered. Without that, a scan would skip the reuse branch, delete the live Chrome's `SingletonLock` , spawn a second Chrome that immediately hands off and exits, time out waiting for CDP, and silently fall back to bundled Chromium with none of your cookies. That is the exact chain that made LinkedIn and Naukri behave as though you were logged out while your real Chrome held valid session cookies the whole time.

Two user-data-dirs, on purpose

Chrome allows exactly one process per user-data-dir. If a background scan held the main directory, clicking "Open & log in" mid-scan would find that process and hand your login window to a *headless* browser — you would click the button and nothing would ever appear. Scans therefore run in their own directory, seeded with a copy of your cookies, re-synced (throttled to once a minute) so a sign-in you complete after the scan started still reaches it.

The stealth script

Eight categories, injected as an init script. The first is the one the rest depend on:

1. **Native-code masking.** Every patched property replaces a native getter with a JS function, and detectors read `Object.getOwnPropertyDescriptor(navigator, 'webdriver').get.toString()` . Real Chrome returns `"function get webdriver() { [native code] }"` ; an unmasked patch returns `"() => undefined"` — an instant, unambiguous flag. A Proxy over `Function.prototype.toString` returns a native-looking string for registered functions, and registers *itself*, so `toString.toString()` also looks native.
2. **navigator.webdriver — patched, not deleted.** Real non-automated Chrome *has* the property and returns `false` , so `('webdriver' in navigator)` is true. Deleting it makes that check false, which no real Chrome ever does: deleting swaps one bot signal for a different one.
3. **Plugins and mimeTypes — built once and cached.** An earlier implementation rebuilt the `PluginArray` on every access, so `navigator.plugins[0] === navigator.plugins[0]` evaluated to `false` — impossible in a real browser and trivially detectable.
4. **Locale and hardware consistency**, because 0 cores or a Linux platform under a Mac UA is a mismatch.
5. **window.chrome** with plausible `app` , `csi` , `loadTimes` and `runtime` surfaces.
6. **Permissions.query** , because headless Chrome famously returns `'denied'` for notifications while `Notification.permission` says `'default'` — a contradiction that is a well-known tell.
7. **WebGL vendor/renderer**, because software-rendered Chrome reports "Google SwiftShader" or "Mesa OffScreen".
8. **Scrubbing CDC and driver artifacts** that leak onto `window` .

`scripts/test-stealth.mjs` asserts 23 fingerprint probes.

MEASURED

Two headless defaults nobody overrode mattered more than any of the above. Same stealth script, headless versus headful: the user agent read `...HeadlessChrome/151...` instead of `...Chrome/151...`, and `screen` was 800×600 instead of 1280×960. With those uncorrected, Naukri's Akamai returned "Access Denied" and Reddit returned zero posts, while the identical code in a visible window got 24 and 10. The real user agent is now read from the installed browser's own `--version` output.

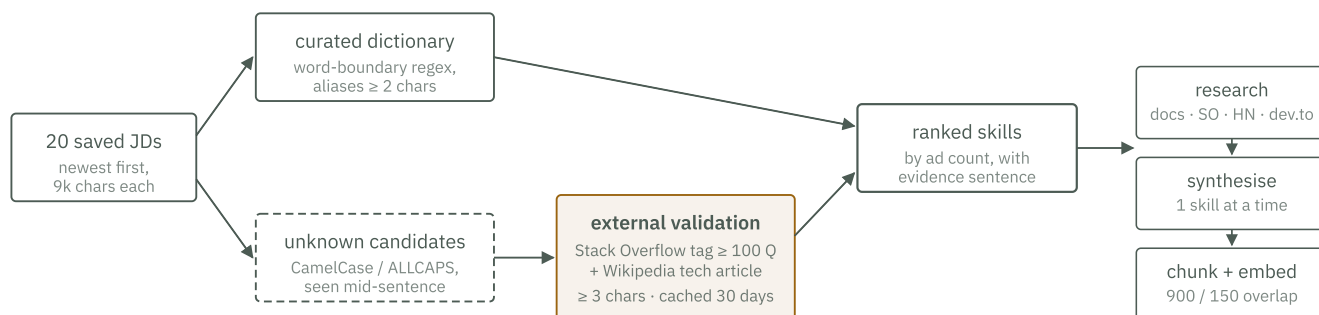
Safe mode

If a human-verification check starts escalating, Safe Mode snapshots every open tab's URL, closes the automated context (cookies flush to disk), launches a brand-new Chrome on the *same* user-data-dir with no debugging port and no init scripts, and reopens the tabs. Logins, cookies and any `cf_clearance` carry over because they live in the directory, not the process. There is literally nothing to detect because nothing is attached.

SECTION 12

Interview prep: from evidence to notes

The prep engine's constraint is that it may not invent study topics. Every topic on the list must trace back to a verbatim sentence in a job description the user actually saved.



Every note carries the JD sentence that put it on the list, how many of your saved jobs demand it, and a numbered reference list — so the user can always check the system's work.

Two paths into the skill list, one validation gate. The curated dictionary provides precision; discovery provides recall for tooling too new to be in it. Everything discovered has to be recognised by an outside authority before it can cost the user forty minutes of local generation.

Why the dictionary is closed

There is deliberately no "guess at unknown capitalised words" fallback. That fallback existed and was a disaster: German job ads capitalise every noun, so it produced study topics like *Du* (you), *Deine* (your), *Kunden* (customers), *Rolle* (role) and *Umsetzung* (implementation). An English stopword list cannot fix

this, because the problem is unbounded — every language, every proper noun, every product name in every ad.

Precision beats recall here for an asymmetric-cost reason: a missed niche tool costs one topic; a garbage topic costs the user's trust and roughly five minutes of local LLM time. Broad fields are excluded too — "study AI" is not an actionable topic, "study RAG" is.

The mid-sentence rule

The structural defence against prose words entering the list is not a wordlist. It is this: *a term must appear capitalised at least once mid-sentence*. "The" and "This" are capitalised only at sentence start; real product names like "Weaviate" or "Kubernetes" also appear capitalised mid-sentence. Nothing else can cover this class — Stack Overflow genuinely has a `this` tag with 6,000+ questions (the JavaScript keyword), so external validation alone cannot reject it. A minimum length of three characters closes the other hole: every real two-character technology (Go, R, C) is already curated, while `du` is both a German pronoun and a Unix command with a real tag.

Research has a relevance gate

Blindly taking top API results meant a search for "Incident Response" returned "Android: Overlay on Android Camera Preview" from Stack Overflow. Every candidate must now mention the skill in its title. The gate splits a multi-word skill into a *primary distinctive term* and secondary tokens, and maintains a `GENERIC` set for the filler inside skill names — matching only "protocol" out of "Model Context Protocol" let "Why is HTTP2 a binary protocol?" through.

Each skill also leads with its **official documentation**, curated alongside the alias list, and sources are tagged by purpose — *learn here* versus *interview questions* versus *background discussion* — because random forum threads do not teach a topic.

Synthesis runs one skill at a time

Two concurrent generations against a single 8B model at `num_ctx: 8192` is what wedged a build: both connections open, model loaded, runner at 3.7% CPU, generating nothing, with no timeout to recover from. The GPU is the bottleneck anyway, so parallelism bought little and risked a stall. Sequential is slower on paper and finishes more often. A full refresh is roughly forty minutes on an 8B model, and each note is written as it completes rather than at the end.

The prep KB is a separate index

Notes are split into 900-character chunks with 150-character overlap, preferring paragraph or sentence boundaries when the break lands past the halfway mark. Re-synthesis deletes a note's chunks before re-indexing so stale material never lingers. The Q&A bot answers strictly from retrieved chunks and says when the KB does not cover something, rather than free-associating — the 0.52 threshold discussed in §6 is what makes that refusal reliable.

The mock interview is deliberately **stateless**: the client holds the transcript and posts it back. No session table, no expiry, and the user can close the tab and resume later. Questions are drawn from the prep KB, so what is rehearsed is what the user's own saved ads actually demand.

The scheduler's four brakes

The scheduler is the one thing in JobFinder that acts on its own, and when armed it sends real applications to real employers while nobody is watching. The failure that matters is "it ran all night", and four independent mechanisms exist to prevent it.

1. **A daily cap counted from the database.** `sentToday()` is a `COUNT(*)` over jobs with `auto_apply_state = 'applied'` since local midnight. A restart, a second window or a crash mid-run cannot reset it, because the number is derived rather than held in memory. The cap also clamps the current batch: `limit = min(limit, cap - sent)`, so no single batch can carry past it.
2. **One run at a time per profile.** A batch that overruns its interval must not have a second one started on top of it, both driving the same browser.
3. **A timeout chain, not `setInterval`.** A slow run delays the next one instead of stacking. The timer is re-armed *before* the run executes, so a thrown run cannot silently end the schedule, and it is `unref`ed so a scheduler is never the reason a server refuses to exit.
4. **Loud, explicit arming.** Only the literal boolean `true` arms it. The schedule announces itself in the log on every server start, naming whether it will send for real — a background sender that starts quietly is exactly what you would want to have been told about.

The schedule *is* persisted, armed state included — a deliberate exception to the rule that arming is never remembered, because a scheduler you must re-arm after every restart is not a scheduler. The daily cap is what makes that survivable. On boot, the first run is delayed by up to five minutes rather than fired immediately, so a server restarting a few times while you edit settings does not send a burst of applications, one per restart.

WHY A SCHEDULED RUN ALSO SCANS

Applying without scanning empties the pool and then does nothing, which reads as "it can't find any new jobs" when the truth is that nothing went looking. Measured overnight: **145 applications between 20:00 and 02:00**, after which Naukri had three candidates left and every later run was a no-op. Scheduled runs now scan first — but at most hourly, because a LinkedIn scan walks four feeds and takes about 85 seconds, so scanning every 20 minutes would spend most of the schedule scanning. The scan is timestamped whether it succeeded or not, so a board that is refusing us is not retried every twenty minutes.

Pipeline, mail and contacts

Stages that cannot slide backwards

Ten stages from `new` to `offer` / `rejected`. `shouldAdvance(current, next)` compares rank, so opening a job you already applied to does not knock it back to "in progress" and lose the applied date. The three terminal-retry stages (`rejected`, `skipped`, `error`) rank `-1` and sit outside the progression: reopening one is a deliberate retry, not a regression.

Funnel diagnosis, deliberately conservative

The conversion rates are the point of the tracker. With a near-zero reply rate the fix is the CV, not more applications — and that is a very different week's work. So `diagnose()` reads the numbers in plain language, and refuses to diagnose small samples: under ten applications it says so and asks for 15–20 before judging. Above that it distinguishes a volume problem (healthy response rate, too few applications) from a quality problem (zero or sub-5% response) from an interview-performance problem (converting to interviews but not offers, with at least three interviews to go on).

Mail: narrow by construction

The original design was paste-only, explicitly to avoid storing a credential in an app that already holds the CV, the answers and the browser sessions. Mailbox connection was added later under a constraint that preserves the spirit: **every sync runs a search built from the user's own applied companies**. Unrelated mail is never requested, downloaded or stored. OAuth uses the read-only `gmail.readonly` scope; an app password grants more than that, but the code only ever reads. Credentials live in `data/gmail.json` at mode 0600, gitignored and revocable.

Job matching is *lexical, not semantic* — looking for the company name in the text, which is reliable and instant. Ambiguity is returned rather than guessed at, because silently updating the wrong application is expensive. Classification is one LLM call per message and the verdict is stored, so re-syncing is cheap; the classifier extracts an `asks` array so the specific things HR wants are listed separately rather than buried in a paragraph.

The contact directory

Scope is deliberately narrow: *only text already fetched for a posting is read*. Nothing is crawled to find addresses, and no address is ever guessed from a name-and-domain pattern. An address gets in because the employer printed it in their own ad next to "send your CV to", which is an invitation to write to them.

MEASURED

Across **1,232 saved postings**, **2.4% contain an address** — and 25 of the 29 come from Hacker News "Who is hiring" threads, where a contact line is the convention. This is a slow-filling, high-quality list rather than a bulk one, which is the right shape for outreach anyway.

Two categories are filtered out because both showed up in real data and both are useless — or rude — to cold-mail. **Job seekers:** Hacker News runs "Who wants to be hired?" alongside "Who is hiring?", and those posts carry the same shape, an address plus *"Location: ... Remote: ... Technologies: ..."*. **Bench-sales and staffing vendors** pitching consultants ("available immediately for C2C", "bench consultants", "hotlist") are selling, not recruiting. The seeker pattern deliberately has *no* trailing `\b` on its alternation: an earlier version ended with one, which silently broke `resume:\s*http` — the character after "http" is "s", not a boundary — so every "Resume: https://..." line slipped through.

SECTION 15

Packaging

`npm run build:desktop` produces a standalone application for macOS and Windows: a real window with Chromium and the Node runtime inside it, which installs the local model runtime on first launch. The recipient double-clicks; there is no terminal, no `npm install`, and no localhost URL to type.

Two structural verifiers guard the outputs — `npm run verify:desktop` runs 23 checks on the desktop packages, `verify:windows` runs 22 on the Windows bundle. These are structural, not functional: they assert that the files that must be present are present and correctly shaped, which is the class of packaging bug that only appears on someone else's machine.

The bundled Chromium is set as `JOBFINDER_BROWSER_FALLBACK`, never as `JOBFINDER_BROWSER_PATH`. Forcing it would discard the better option on every machine that has real Chrome installed, and a lived-in browser profile is worth more to bot-detection outcomes than anything the stealth script does.

SECTION 16

Where the ML actually is

JobFinder trains nothing. There is no model file, no fitted parameters and no learning loop. What it uses are two pretrained models — one embedding, one generative — plus rule systems that are frequently mistaken for models.

COMPONENT	WHAT IT IS	WHY NOT SOMETHING LEARNED
Answer retrieval	all-minilm sentence embeddings (384-dim), cosine similarity, top-K with a threshold. A pretrained bi-encoder used zero-shot.	Nothing to train on: a personal answer bank has 35 rows, not 35,000. Zero-shot semantic similarity is exactly the right tool at that scale, and the only tuning needed was <i>choosing the encoder</i> and <i>setting the threshold</i> — both done by measurement (§6).
Prep Q&A	The same encoder over a separate chunk index — retrieval-augmented generation with a hard grounding rule.	Same reason, plus a hard requirement that answers cite the notes rather than the model's own priors.
Answer drafting, cover letters, synthesis, classification	A local instruct LLM (qwen2.5:7b-instruct) prompted with retrieved context. Constrained JSON decoding where structure matters.	Generation, not prediction. The model is used where the task is genuinely a writing task and refused where the task is a factual claim about the user.
Fit scoring	A hand-weighted linear rule model over four components. <i>Not</i> a classifier.	Three reasons: no labels (nobody annotates "was this a good job for me"), a hard latency requirement (hundreds of jobs, sub-second, on CPU), and an explainability requirement — the user has to be able to read <i>why</i> a job scored 72 and disagree with it.
Skill extraction	A curated dictionary of regexes plus an external-authority validator (Stack Overflow tags, Wikipedia).	A learned NER model would be the textbook answer and would produce exactly the German-noun failure the dictionary was written to avoid. The validator is a cheap, auditable substitute for a model with a confidence score nobody could interpret.
CV variant selection	Deterministic set coverage — the variant whose extracted text matches the most of the posting's required skills wins.	Picking a file must be instant and reproducible, and the user has to be able to see why it chose what it chose. The log prints the winner's coverage percentage and every runner-up's.
Salary and notice period	Arithmetic over published constants (World Bank PPP) and calendar dates.	These have correct answers. Estimating them would be strictly worse than computing them.
Every component that could plausibly be called "ML", what it actually is, and why that choice.		

The through-line

The pattern across all seven rows is the same test, applied consistently: **is the output a claim about the world, or a piece of writing?** Writing goes to the model. Claims — how many years of experience you have, what your notice period is, which currency the form wants, whether a term is a real technology — go to something deterministic, auditable, or to the user.

The one place that test is enforced at runtime rather than at design time is the [BLANK] sentinel: the short-field prompt instructs the model to return the literal string [BLANK] rather than invent a phone number, address, salary, visa status, date or employer name, and the caller treats that string as "leave it empty". It is a small mechanism carrying a large policy — and in auto-apply, where the stakes are highest, the model is not given the opportunity at all.

Generated from the JobFinder source tree — `lib/`, `connectors/`, `app/api/`, `desktop/`. Measured figures quoted throughout are those recorded in the source comments at the time each decision was made. MIT licensed; respect each site's terms of service.